Informatica PowerCenter - Retail Data Warehouse ETL

INFORMATICA POWERCENTER + POSTGRESQL | DATA ENGINEERING PORTFOLIO

Project Explanation and Build Guide

Technology: Informatica PowerCenter, PostgreSQL, SQL, CSV **Project type:** Data engineering / ETL / dimensional data warehousing **Repository:** https://github.com/shubham05jaiswal-coder/informatica-retail-data-warehouse

> **Scope:** This project is an implementation blueprint with sample data, PostgreSQL DDL, mapping specifications, workflow design, and analytical SQL. It is not an exported, import-ready Informatica repository package. Build and validate PowerCenter objects in your own licensed/version-specific environment.

1. Executive summary

The project combines four retail data feeds - customers, products, order headers, and order lines - into a structured warehouse that supports sales reporting. Informatica PowerCenter is the planned ETL engine. PostgreSQL is the reference target database and reporting layer.

The objective is not just to copy rows. The ETL process should standardize fields, validate values, resolve relationships between source keys, calculate sales measures, route invalid rows to reject handling, protect the warehouse against duplicate reruns, and record the result of each load batch.

Business questions the warehouse can answer include monthly net sales, units sold, best-selling products, customer sales value, and the frequency and causes of data-quality rejects.

2. Architecture and data flow

```
CSV files
(customers, products, orders, order_items)
             |
             v
Informatica PowerCenter
Source Qualifier -> Expression -> Lookup -> Router
             |
             v
PostgreSQL staging tables
             |
             v
Customer / Product / Date dimensions
             |
             v
Sales fact table (one row per accepted order item)
             |
             v
SQL analytics / BI reporting

Reject logging and batch monitoring apply across the pipeline.
```

CSV sources: small sample flat files represent independent retail source systems. **PowerCenter mappings:** read, clean, validate and route each source row. **Staging:** temporarily land validated source records with run/file metadata, separate from final warehouse loading. **Dimensions:** hold descriptive entities such as customers, products and calendar dates. **Fact table:** stores sales events and numeric measures at a clearly defined grain. **SQL layer:** aggregates the fact table for business reporting and checks batch/reject outcomes.

3. Input files and business keys

File	Row grain	Main columns
`customers.csv`	One customer	**`customer_id`**, first/last name, email, city/state, signup date
`products.csv`	One product	**`product_id`**, product name, category, unit price
`orders.csv`	One order header	**`order_id`**, **`customer_id`**, order date, status, payment method
`order_items.csv`	One order line	**`order_item_id`**, **`order_id`**, **`product_id`**, quantity, price, discount

Important relationships:

- **`orders.customer_id`** references **`customers.customer_id`**.
- **`order_items.order_id`** references **`orders.order_id`**.
- **`order_items.product_id`** references **`products.product_id`**.
- **`order_item_id`** identifies the source line and should prevent duplicate fact rows when the same batch is rerun.

The sample dataset is intentionally small for review and includes quality problems to exercise validation logic.

4. Data-quality scenarios

Scenario	Sample case	Expected design response
Duplicate customer	Customer C006 appears twice.	Define a deterministic duplicate policy, record counts, and avoid violating dimension business-key uniqueness.
Invalid date	C007 has a malformed signup date.	Parse using an explicit date mask; route the invalid value to reject handling or quarantine.
Unknown product	An order line references product P999, which is not present in the product file.	A product lookup fails; reject the row or use an explicitly agreed unknown-member policy.
Invalid quantity	An order line has quantity -1.	Reject the line because quantity must be greater than zero.
Order status	Sample orders include COMPLETED, CANCELLED and PROCESSING.	Confirm business rules for which statuses contribute to recognized sales. The sample analytics exclude CANCELLED orders.

Different failures should have different reason codes. A bad date is a parse failure; an unknown product is a referential-integrity failure; a cancelled order is a business-status decision rather than automatically a malformed source row.

5. Main PowerCenter mappings

A mapping specifies the source-to-target movement and the transformations applied to each row. The repository has seven mapping design specifications.

Mapping	Responsibility	Main logic
`m_load_stg_customers`	Prepare customer records.	Trim/normalize fields, parse signup dates, validate required values, route valid/reject rows.
`m_load_stg_products`	Prepare product master records.	Convert unit price, validate product key/name and price, normalize category.
`m_load_stg_orders`	Prepare order headers.	Parse order dates, standardize status/payment method, validate customer references.
`m_load_stg_order_items`	Prepare sales lines.	Cast line ID/quantity/price, validate quantity and discount, lookup order and product.
`m_dim_customer_scd1`	Maintain the customer dimension.	Lookup by customer business key; insert new, update changed, skip unchanged. SCD Type 1 overwrites current attributes and does not preserve prior history.

Mapping	Responsibility	Main logic
`m_dim_product`	Maintain product dimension.	Lookup by product business key and apply deterministic insert/update logic.
`m_fact_sales`	Load sales fact rows.	Resolve date/customer/product surrogate keys, calculate measures, apply status rules and prevent duplicate lines.

Key PowerCenter transformations

- **Source Qualifier:** provides source rows from the flat-file source definition.
- **Expression:** trims and standardizes data, converts datatypes, validates row-level rules, and calculates amounts.
- **Lookup:** validates a customer/order/product relationship or resolves the appropriate dimension surrogate key.
- **Router:** separates valid rows, rejected rows and any special handling groups.
- **Aggregator:** use only where a grouped calculation is required; the sales fact remains at order-line grain in this design.
- **Update Strategy and target settings:** control insert/update behaviour. Combine them with database constraints and a defined rerun strategy.

The mapping names in this repository are logical design names. Confirm expression syntax, port datatypes, connection objects, and session settings against your installed PowerCenter version.

6. Dimensional model

Table	Purpose	Important key design
`dw.dim_customer`	Customer attributes such as name, email, city, state and signup date.	`customer_key` is a surrogate key; `customer_id` is the source business key.
`dw.dim_product`	Product attributes such as product name, category and unit price.	`product_key` is a surrogate key; `product_id` is the source business key.
`dw.dim_date`	Calendar attributes used for month, quarter and year reporting.	`date_key` uses YYYYMMDD format and relates to the calendar date.
`dw.fact_sales`	One accepted order line with sales measures and dimension keys.	`order_item_id` is unique; customer, product and date keys are foreign keys.
`stg.*`	Staging tables used between ingestion and dimensional loading.	Keep run/file metadata so records can be traced to an input batch.
`etl.reject`	Invalid/unresolved records and reason details.	Include run ID, source name, business key, raw row and reason code/detail.
`etl.batch_control`	Workflow-level execution audit.	Track status, start/end timestamps, source/loaded/rejected counts and message.

Fact grain: one row in `dw.fact_sales` represents one validated source order item. Define grain before building the mapping because it determines the measures, keys and duplicate-prevention strategy.

Sales calculations

The CSV sample represents `discount_pct` as a percentage from 0 to 100 (for example, 5 means five percent). Divide by 100 in the calculation:

```
gross_amount       = quantity * unit_price
discount_amount    = gross_amount * (discount_pct / 100)
net_sales_amount   = gross_amount - discount_amount
```

Example: 2 units at 799.00 with a 5% discount produce gross sales of 1,598.00, a 79.90 discount, and net sales of 1,518.10.

Load dimensions before the fact mapping so that valid dimension surrogate keys are available to the fact rows.

7. Daily workflow and operational controls

The design named `wf_retail_dw_daily` coordinates a recommended sequence:

1. Start the batch and allocate a unique run ID.

2. Load customer and product staging data (in parallel if appropriate).
3. Load order headers after customer data is available.
4. Load order items after order/product references can be checked.
5. Load customer and product dimensions and ensure date dimension rows exist.
6. Load validated, eligible order lines into the fact table.
7. Reconcile source, accepted, loaded and rejected counts.
8. Mark the batch successful or failed and retain diagnostic logs.

Operational controls to implement:

- **Batch/run ID:** correlate source rows, staging rows, rejects and audit rows with one execution.
- **Count reconciliation:** account for loaded, rejected and duplicate-handled rows using a documented equation.
- **Reject logging:** retain source name, business key, reason code/detail and raw source row when appropriate.
- **Restart safety:** enforce uniqueness and use deterministic load rules so reruns do not duplicate facts.
- **Failure handling:** connect failure paths, mark the batch failed, and preserve session/workflow logs.
- **Security:** configure secrets through secure connection objects or secret management. Never commit database passwords to GitHub.

8. Setup and test plan

1. Prepare a PostgreSQL database and review **`sql/01_create_schema.sql`**.
2. Populate the calendar table with **`sql/03_load_date_dimension.sql`**.
3. Register the four CSV files as PowerCenter flat-file sources and define/import PostgreSQL targets.
4. Build the four staging mappings, including validation and reject paths.
5. Build customer Type 1 and product dimension mappings; verify business-key uniqueness.
6. Build **`m_fact_sales`** with lookups to dimensions, amount calculations, order-status rules and rerun protection.
7. Create sessions and the daily workflow using **`informatica/workflow_specs/wf_retail_dw_daily.md`**.
8. Test good inputs plus duplicate customer, malformed date, unknown product, negative quantity, missing source file, database connection failure, and rerunning a batch.
9. Run **`sql/02_analytics_queries.sql`** and reconcile the results to the fact table and audit counts.

9. Analytics included

The SQL scripts demonstrate:

- Monthly net sales, orders and units sold.
- Top products ranked by net sales and unit volume.
- Customer cumulative sales value and order counts.
- Reject monitoring by reason.
- Recent batch status, row counts and timings.

These queries are examples for project validation and reporting; adapt filter rules, date ranges, status semantics, and KPI definitions to the business requirement.

10. Interview explanation

I designed a retail data warehouse ETL blueprint using Informatica PowerCenter and PostgreSQL. Four CSV files provide customers, products, order headers and order lines. PowerCenter mappings standardize fields, validate dates and measures, use lookups to confirm relationships, and route invalid records to reject tracking. Valid rows land in staging before customer/product/date dimensions and an order-line-grain sales fact are loaded. The fact mapping resolves surrogate keys and calculates gross and net sales. A daily workflow coordinates dependencies, records batch status and row counts, and supports failure diagnosis and safe reruns. SQL queries report monthly sales, best-selling products, customer sales value and data-quality outcomes. The repository contains the implementation blueprint; the PowerCenter mappings, sessions and workflow still need to be built and tested in the actual environment.

11. Important implementation alignment notes

Before running the project, align physical table names across the PostgreSQL DDL, source-to-target mapping CSV and mapping specs. The current SQL uses schema-qualified names such as **`stg.customers`**, **`etl.reject`** and **`etl.batch_control`**, while some design documents use logical labels such as **`stg_customers`** or **`etl_reject`**. Map the logical names to the chosen physical names consistently.

Confirm the discount convention across data, validation and formulas. The supplied rows use 0, 5 and 10 and the SQL reference DDL allows 0 through 100, so divide by 100. If adopting fractional values such as 0.05, update the sample data, validation rules, column checks and formulas together.

Finally, clarify whether PROCESSING orders count as recognized sales. The supplied analytics exclude CANCELLED orders, but PROCESSING treatment should be agreed with stakeholders and applied consistently in fact loading and reporting filters.

Repository layout

- **`data/source/`** - sample CSV files
- **`sql/`** - PostgreSQL schema, date dimension and analytics
- **`informatica/mapping_specs/`** - mapping designs
- **`informatica/workflow_specs/`** - daily workflow design
- **`docs/source_to_target_mapping.csv`** - source-to-target lineage
- **`docs/data_dictionary.md`** - key definitions and quality rules
- **`docs/powercenter_build_checklist.md`** - implementation/acceptance checklist

Open the GitHub repository

A GitHub Actions workflow builds the companion PDF from this Markdown guide whenever the guide or its generator changes.